

✅ 📅 DAY 3 — HYBRID RETRIEVAL (WITH WHY + CODE)

🎯 WHY WE ARE CHANGING (IMPORTANT)

❌ Your Day 2 system:

Query → Vector → Result

👉 Problem:

- good for meaning ✅
- bad for exact words ❌

Example:

Query: "engine fault code 123"

Vector might miss exact "123" ❌

✅ So we ADD BM25:

BM25 → matches keywords ✅

Vector → matches meaning ✅

✅ Final system:

Query

↓

BM25 (keywords)

Vector (semantic)

↓

RRF → combine both

👉 This is industry standard retrieval

✅ WHAT YOU ARE DOING TODAY

👉 Not replacing ✅

👉 Combining two methods

✅ STEP 1 — BM25 (KEYWORD ENGINE)

📁 retrieval/bm25.py

```
from rank_bm25 import BM25Okapi
```

```
class BM25Retriever:
```

```
    def init(self):
```

```
        self.bm25 = None
```

```
        self.docs = []
```

```
    def fit(self, documents):
```

```
        self.docs = documents
```

```
        tokenized = [doc.split() for doc in documents]
```

```
        self.bm25 = BM25Okapi(tokenized)
```

```
    def search(self, query, top_k=5):
```

```
        scores = self.bm25.get_scores(query.split())
```

```
        ranked = sorted(
```

```
            zip(self.docs, scores),
```

```
            key=lambda x: x[1],
```

```
        reverse=True
    )

    return [doc for doc, _ in ranked[:top_k]]
```

✅ What it does:

- splits text into words
 - finds exact matches
-

✅ STEP 2 — FUSION (COMBINE BOTH)

📁 retrieval/hybrid.py

```
def reciprocal_rank_fusion(bm25_docs, vector_docs, k=60):
    scores = {}

    for rank, doc in enumerate(bm25_docs):
        scores[doc] = scores.get(doc, 0) + 1 / (rank + k)

    for rank, doc in enumerate(vector_docs):
        scores[doc] = scores.get(doc, 0) + 1 / (rank + k)

    sorted_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)

    return [doc for doc, _ in sorted_docs]
```

✅ What it does:

- gives score based on position
 - combines both lists
 - ranks best chunks
-

✅ STEP 3 — HYBRID SEARCH (MAIN CHANGE 🔥)

📁 retrieval/vector_search.py

👉 Replace old class with this:

```
from embeddings.embedding_service import EmbeddingService
from embeddings.vector_store import VectorStore
from retrieval.bm25 import BM25Retriever
from retrieval.hybrid import reciprocal_rank_fusion
```

```
class HybridSearch:
```

```
    def init(self):
```

```
        self.embedder = EmbeddingService()
```

```
        self.store = VectorStore()
```

```
        self.bm25 = BM25Retriever()
```

```
        self.documents = []
```

```
    def index(self, documents):
```

```
        self.documents = documents
```

```
    # ✅ Vector index
```

```
embeddings = self.embedder.encode(documents)
```

```
ids = [str(i) for i in range(len(documents))]
```

```
metadata = [{"source": "file"} for _ in documents]
```

```
self.store.add(ids, documents, embeddings, metadata)
```

```
#  BM25 index
```

```
self.bm25.fit(documents)
```

```
def search(self, query):
```

```
    #  Vector search
```

```
    query_embedding = self.embedder.encode(query)[0]
```

```
    vector_results = self.store.query(query_embedding)
```

```
    #  BM25 search
```

```
    bm25_results = self.bm25.search(query)
```

```
    #  Combine both
```

```
    final_results = reciprocal_rank_fusion(
```

```
        bm25_results,
```

```
        vector_results
```

```
    )
```

```
    return final_results[:5]
```

✅ What changed:

- earlier → only vector
- now → vector + bm25 + fusion

✅ STEP 4 — UPDATE MAIN

📁 app/main.py

👉 Change ONLY this:

```
from retrieval.vector_search import HybridSearch  
  
and  
  
vs = HybridSearch()
```

✅ FINAL FLOW NOW

User query



Vector search ✅

BM25 search ✅



Fusion ✅



Best chunks ✅



Qwen 🔥

✅ WHAT YOU GAIN

Before	After
semantic only	semantic + keyword
misses numbers	captures numbers
weaker search	strong search ✅

🧠 SIMPLE ANALOGY

- Vector = understands meaning
- BM25 = finds exact words

👉 Hybrid = both brain + dictionary

✅ TEST

Run:

```
python -m app.main
```

Try:

```
>> what is machine learning
```

```
>> deep learning meaning
```

✅ RESULT

👉 Answers are:

- more relevant ✅
 - more precise ✅
 - stable ✅
-

FINAL DAY 3 STATUS

 You now built:

Hybrid RAG system 

 This is already intermediate/industry level

NEXT

Say:

 "Day 4"

We'll add:

-  reranker (huge quality jump)
 - final refinement layer
-

 And good question — this "why" mindset is exactly how you become strong in AI systems.